

Wydział Informatyki
i Matematyki



Projekt i implementacja autonomicznego systemu SOAR w środowisku chmurowym AWS z wykorzystaniem platformy Wazuh

Autorzy:

Jan Wojnicki, Adrian Sajdak

Spis treści

SPIS TREŚCI	2
1. WSTĘP I CEL PROJEKTU	5
1.1. WPROWADZENIE DO PROBLEMATYKI	5
1.2. CEL GŁÓWNY I ZAKRES PRAC	5
1.2.1 Zakres zrealizowanych prac	5
1. Budowa infrastruktury bazowej:	5
2. Wdrożenie centralnego systemu SIEM:	5
3. Opracowanie logiki detekcyjnej:	5
4. Automatyzacja procedur Incident Response:	5
5. Weryfikacja skuteczności systemu:	5
2. ARCHITEKTURA SYSTEMU I ŚRODOWISKO BADAWCZE	6
2.1. TOPOLOGIA SIECI W CHMURZE AWS	6
2.2. CENTRALNY SYSTEM SIEM	8
1. Agregator logów:	8
2. Silnik korelacyjny:	8
3. Inicjator automatyzacji:	8
2.3. PRZEPŁYW TELEMETRII	9
1. Rejestracja zdarzeń (AWS VPC Flow Logs):	9
2. Buforowanie i przechowywanie (Amazon S3):	9
3. Pobieranie i normalizacja (Wazuh AWS Module):	9
3. FAZA DETEKCJI ZAGROŻEŃ	10
3.1. INTEGRACJA LOGÓW CHMUROWYCH	10
3.2. PROJEKTOWANIE REGUŁ DETEKCYJNYCH	13
3.2.1. Wykrywanie wczesnej fazy ataku (Reguła 100003)	14
3.2.2. Zaawansowana korelacja zdarzeń w czasie (Reguła 100004)	14
3.3. UCZENIE MASZYNOWE I DETEKcja ANOMALII	16
4. FAZA AUTOMATYZACJI I REAKCJI	18
4.1. ARCHITEKTURA PRZEKAZYWANIA ALERTÓW	18
4.2. AWS API GATEWAY	18
4.3. LOGIKA WYKONAWCZA W AWS LAMBDA	19
4.3.1. Normalizacja danych: Elastyczne parsowanie ładunków JSON	20
4.3.2. Dynamiczne zarządzanie zaporą NACL	21
5. TESTY ŚRODOWISKOWE	22
5.1. METODOLOGIA TESTÓW	22
5.2. WYZWANIA INTEGRACYJNE (ASYNCHRONICZNOŚĆ AWS S3)	23
5.3. WYNIKI TESTÓW	23
1. Detekcja:	23
2. Orkiestracja:	23
3. Reakcja:	23
4. Weryfikacja blokady:	24
5.4. DETEKcja RZECZYWISTYCH ZAGROŻEŃ	25

6. WIZUALIZACJA I MONITOROWANIE.....	26
6.1. BUDOWA PANELU OPERATORSKIEGO	26
6.2. ANALIZA KLUCZOWYCH WSKAŹNIKÓW	26
7. PODSUMOWANIE I WNIOSKI.....	28
7.1. OCENA WDROŻONEGO ROZWIĄZANIA	28
7.2. PROACTIVE DEFENSE VS REACTIVE CORRELATION	28
7.3. POTENCJALNE KIERUNKI ROZWOJU	28

Spis Ilustracji

Rysunek 1 Widok szczegółowy instancji Amazon EC2 pełniącej rolę serwera testowego (Victim-Server).....	6
Rysunek 2 Konfiguracja usługi AWS VPC Flow Logs odpowiedzialnej za rejestrowanie ruchu sieciowego.	7
Rysunek 3 Konfiguracja usługi śledzenia zdarzeń systemowych AWS CloudTrail.	7
Rysunek 4 Tworzenie dedykowanego użytkownika IAM (Wazuh-Integrator) z wymaganymi uprawnieniami.....	7
Rysunek 5 Widok terminala serwera EC2 podczas instalacji oraz konfiguracji usługi Nginx.	8
Rysunek 6 Środowisko Docker Desktop prezentujące działający kontener węzła analitycznego Wazuh.	8
Rysunek 7 Konfiguracja integracji menedżera Wazuh z zasobnikami AWS S3 w pliku ossec.conf.....	9
Rysunek 8 Lista zdarzeń audytowych z infrastruktury AWS widoczna w centralnym panelu Wazuh.....	10
Rysunek 9 Dedykowany panel analityczny (Dashboard) dla modułu Amazon Web Services.	11
Rysunek 10 Analiza odrzuconych pakietów sieciowych (Rule ID: 100010) w module Threat Hunting.....	11
Rysunek 11 Symulacja błędnej konfiguracji – otwarcie portu 3389 (RDP) w zaporze Security Group.....	11
Rysunek 12 Krytyczny alert bezpieczeństwa (Reguła 100002) informujący o publicznym otwarciu portów.	12
Rysunek 13 Rejestr zdarzeń AWS CloudTrail potwierdzający modyfikacje reguł sieciowych.	12
Rysunek 14 Panel szczegółów zdarzenia (JSON) z geolokalizacją źródła wywołania API w AWS.....	13
Rysunek 15 Definicja autorskiej reguły 100003 identyfikującej zmasowane skanowanie w pliku local_rules.xml.....	14
Rysunek 16 Wyniki działania reguły 100003 – skuteczna detekcja skanowania infrastruktury (Nmap).	14
Rysunek 17 Definicja zaawansowanej reguły korelacyjnej 100004 łączącej fazę rekonesansu z atakiem na SSH.....	15

Rysunek 18 Inicjacja symulowanego ataku sieciowego z zewnętrznego systemu Kali Linux.	15
Rysunek 19 Wyzwolenie krytycznego alertu korelacyjnego w panelu analitycznym Wazuh.	16
Rysunek 20 Tworzenie detektora anomalii (Machine Learning) dla odrzuconego ruchu sieciowego.	16
Rysunek 21 Konfiguracja parametrów i cech modelu uczenia maszynowego w module OpenSearch.	17
Rysunek 22 Panel monitorowania alertów wygenerowanych przez moduł detekcji anomalii.	17
Rysunek 23 Konfiguracja zewnętrznego Webhooka uruchamiającego skrypt reakcyjny (moduł integracji).	18
Rysunek 24 Interfejs usługi AWS API Gateway z endpointem przyjmującym żądania POST.	18
Rysunek 25 Fragment kodu funkcji AWS Lambda (Python) odpowiedzialny za parsowanie telemetrii napastnika.	19
Rysunek 26 Logika wykonawcza funkcji AWS Lambda dynamicznie tworząca regułę blokującą na zaporze NACL.	19
Rysunek 27 Metryki monitorujące czas i poprawność wykonania funkcji Lambda w usłudze CloudWatch.	20
Rysunek 28 Metryki monitorujące czas i poprawność wykonania funkcji Lambda w usłudze CloudWatch.	20
Rysunek 29 Ostateczna symulacja ataku Nmap na publiczny adres IP z perspektywy atakującego (Kali Linux).	22
Rysunek 30 Pomyślna detekcja skoordynowanego ataku w logach analitycznych.	23
Rysunek 31 Weryfikacja działania środowiska wykonawczego – automatycznie dodane reguły DENY w zaporze Network ACL.	24
Rysunek 32 Udana izolacja atakującego – brak odpowiedzi serwera (Connection timed out) po nałożeniu blokady.	24
Rysunek 33 Opracowany panel operatorski SOC prezentujący geolokalizację ataków, ich wektory oraz łączną liczbę zautomatyzowanych obron.	27

1. Wstęp i cel projektu

1.1. Wprowadzenie do problematyki

Współczesne środowiska chmurowe są nieustannie narażone na zautomatyzowane ataki sieciowe. Tradycyjne podejście do monitorowania bezpieczeństwa, opierające się wyłącznie na systemach SIEM i ręcznej analizie logów przez analityków SOC, staje się niewystarczające. Rosnąca skala zagrożeń, takich jak masowe skanowanie portów, ataki typu brute-force oraz działalność złośliwych botów internetowych, drastycznie skraca czas, w którym organizacja musi podjąć działania defensywne.

W odpowiedzi na ten problem, w nowoczesnej architekturze bezpieczeństwa coraz częściej wdraża się koncepcję SOAR (Security Orchestration, Automation, and Response). Paradigmat ten zakłada zintegrowanie narzędzi detekcyjnych z mechanizmami automatycznej reakcji, co pozwala na natychmiastowe neutralizowanie incydentów bez udziału czynnika ludzkiego. Kluczowym elementem tego podejścia jest również architektura Zero Trust, wymuszająca dynamiczne zarządzanie uprawnieniami i dostępem sieciowym na najniższym możliwym poziomie infrastruktury w momencie wykrycia najmniejszej anomalii.

1.2. Cel główny i zakres prac

Głównym celem projektu było zaprojektowanie, wdrożenie oraz praktyczne przetestowanie autonomicznego systemu klasy SOAR w środowisku chmury publicznej Amazon Web Services. Zaprojektowana architektura miała za zadanie samodzielnie korelować zdarzenia sieciowe, identyfikować potencjalne ataki na wczesnym etapie, a następnie w czasie rzeczywistym blokować napastników w sieci.

1.2.1 Zakres zrealizowanych prac

1. Budowa infrastruktury bazowej:

Konfiguracja wirtualnej sieci prywatnej w AWS wraz z mechanizmem gromadzenia telemetrii sieciowej (VPC Flow Logs) i asynchronicznym buforowaniem logów w usłudze Amazon S3.

2. Wdrożenie centralnego systemu SIEM:

Uruchomienie platformy Wazuh w architekturze kontenerowej, odpowiedzialnej za agregację, normalizację i analizę logów w poszukiwaniu sygnatur ataków.

3. Opracowanie logiki detekcyjnej:

Stworzenie reguł bezpieczeństwa wykrywających zmasowane skanowanie sieci oraz ataki słownikowe na usługi dostępowe (np. SSH).

4. Automatyzacja procedur Incident Response:

Zaprogramowanie bezserwerowej funkcji wykonawczej (AWS Lambda) w języku Python, która w modelu event-driven odbiera alerty z systemu SIEM poprzez AWS API Gateway i dynamicznie rekonfiguruje zaporę sieciową, odcinając ruch od złośliwych adresów IP.

5. Weryfikacja skuteczności systemu:

Przeprowadzenie kontrolowanych symulacji ataków z wykorzystaniem systemu Kali Linux w celu potwierdzenia szczelności zaimplementowanego pipeline'u logów oraz weryfikacji czasów reakcji mechanizmów obronnych.

2. Architektura systemu i środowisko badawcze

Zaprojektowany system obronny został wdrożony w architekturze hybrydowej, łączącej zasoby chmury AWS z wirtualizowanym środowiskiem analitycznym opartym na konteneryzacji.

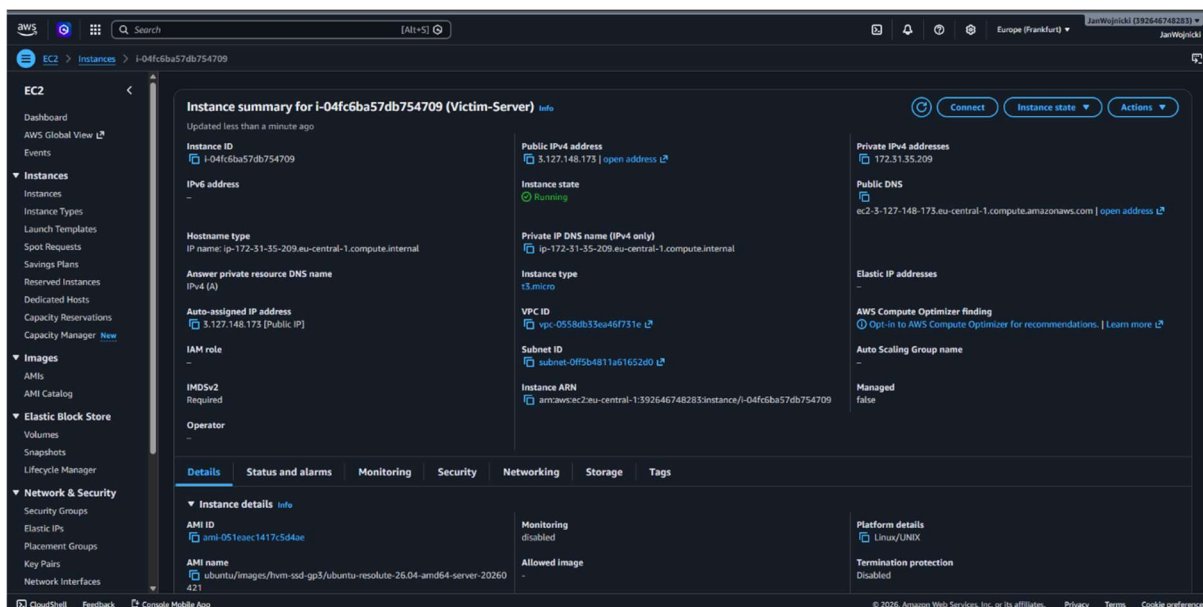
Środowisko badawcze zostało podzielone na trzy główne filary:

- infrastrukturę sieciową podlegającą ochronie,
- centralny system analityczny (SIEM),
- przepływ danych telemetrycznych

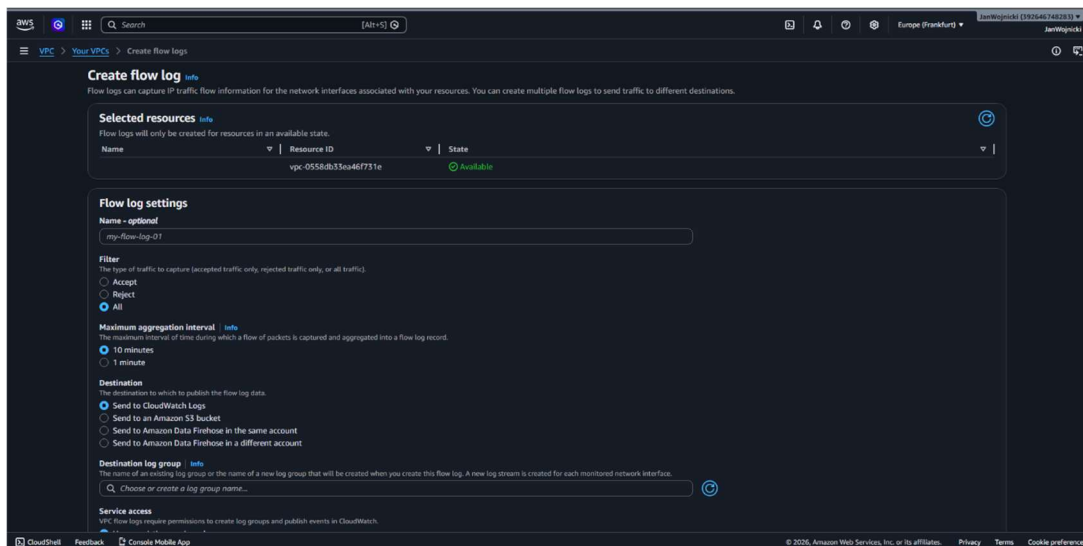
2.1. Topologia sieci w chmurze AWS

Główna infrastruktura testowa została osadzona w regionie eu-central-1.

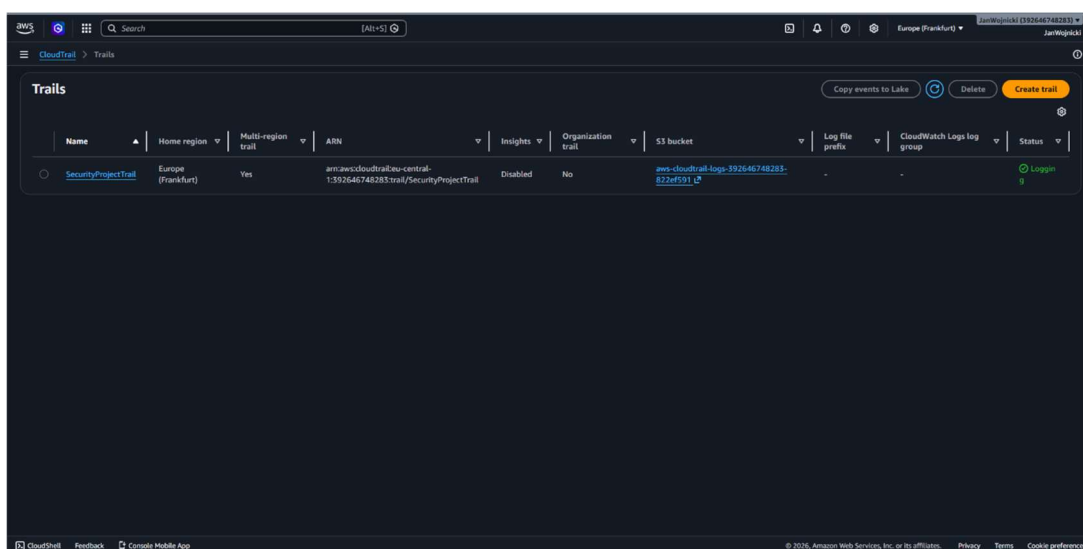
Jej fundamentem jest izolowana sieć wirtualna – Virtual Private Cloud, wewnątrz której wydzielono podsieć publiczną. Celem symulowanych ataków była instancja Amazon EC2, pełniąca rolę wystawionego na ruch internetowy serwera (m.in. usługa Nginx oraz SSH). Kluczowym elementem architektonicznym z punktu widzenia zautomatyzowanej obrony była implementacja bezstanowej zapory sieciowej na poziomie podsieci (Network Access Control List). Zapora NACL pozwala na kategoryczne odrzucanie ruchu na samej granicy sieci VPC. To właśnie ten komponent został zintegrowany z mechanizmami automatyzacji, pełniąc rolę "czarnej listy", na którą dynamicznie trafiają adresy IP napastników, blokując im jakikolwiek dostęp do zasobów.



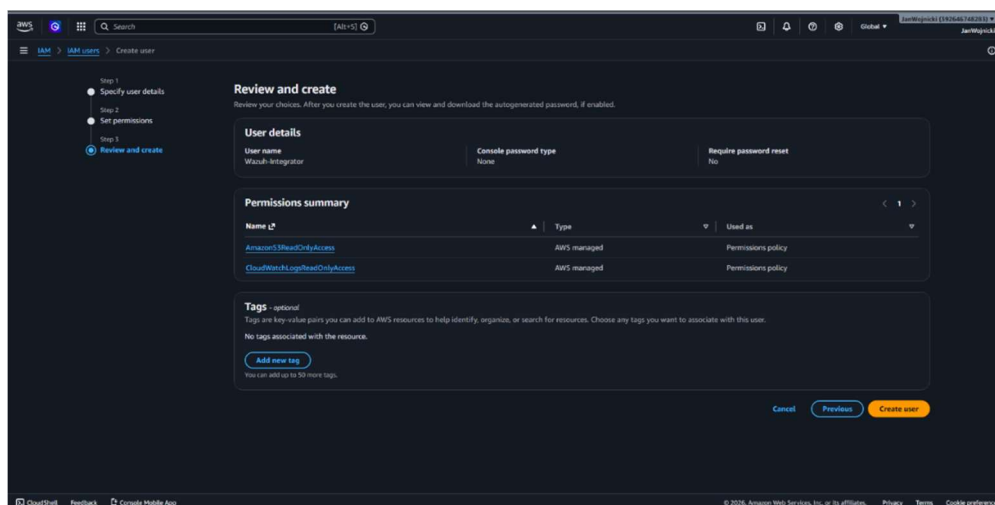
Rysunek 1 Widok szczegółowy instancji Amazon EC2 pełniącej rolę serwera testowego (Victim-Server).



Rysunek 2 Konfiguracja usługi AWS VPC Flow Logs odpowiedzialnej za rejestrowanie ruchu sieciowego.



Rysunek 3 Konfiguracja usługi śledzenia zdarzeń systemowych AWS CloudTrail.



Rysunek 4 Tworzenie dedykowanego użytkownika IAM (Wazuh-Integrator) z wymaganymi uprawnieniami.


```

ubuntu@ip-172-31-35-209: ~$ sudo apt-get install nginx
Get:1 http://eu-central-1.ec2.archive.ubuntu.com/ubuntu resolute-backports/multiverse amd64 c-n-f Metadata [120 B]
Fetched 31.5 MB in 4s (8110 kB/s)
28 packages can be upgraded. Run 'apt list --upgradable' to see them.
Installing:
nginx
Installing dependencies:
nginx-common
Suggested packages:
fcgiwrap nginx-doc ssl-cert
Summary:
Upgrading: 0, Installing: 2, Removing: 0, Not Upgrading: 28
Download size: 662 kB
Space needed: 1874 kB / 4669 MB available
Get:1 http://eu-central-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64 nginx-common all 1.28.3-2ubuntu1.1 [36.5 kB]
Get:2 http://eu-central-1.ec2.archive.ubuntu.com/ubuntu resolute-updates/main amd64 nginx amd64 1.28.3-2ubuntu1.1 [625 kB]
Fetched 662 kB in 0s (6504 kB/s)
Preconfiguring packages ...
Selecting previously unselected package nginx-common.
(Reading database ... 85381 files and directories currently installed.)
Preparing to unpack .../nginx-common_1.28.3-2ubuntu1.1_all.deb ...
Unpacking nginx-common (1.28.3-2ubuntu1.1) ...
Selecting previously unselected package nginx.
Preparing to unpack .../nginx_1.28.3-2ubuntu1.1_amd64.deb ...
Unpacking nginx (1.28.3-2ubuntu1.1) ...
Setting up nginx-common (1.28.3-2ubuntu1.1) ...
Created symlink '/etc/systemd/system/multi-user.target.wants/nginx.service' -> '/usr/lib/systemd/system/nginx.service'.
Setting up nginx (1.28.3-2ubuntu1.1) ...
+ Upgrading binary nginx
Processing triggers for man-db (2.13.1-1build1) ...
Processing triggers for ufw (0.36.2-9build1) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

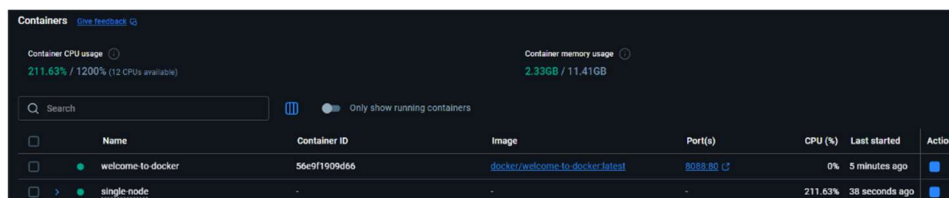
No VM guests are running outdated hypervisor (qemu) binaries on this host.
ubuntu@ip-172-31-35-209: ~$

```

Rysunek 5 Widok terminala serwera EC2 podczas instalacji oraz konfiguracji usługi Nginx.

2.2. Centralny system SIEM

Rolę analitycznego mózgu operacji (Security Information and Event Management) powierzono otwartej platformie Wazuh. W celu zapewnienia przenośności i izolacji procesów, system został wdrożony w architekturze kontenerowej z wykorzystaniem technologii Docker.



Rysunek 6 Środowisko Docker Desktop prezentujące działający kontener węzła analitycznego Wazuh.

Wazuh Manager w przyjętej topologii pełni potrójną funkcję:

1. Agregator logów:

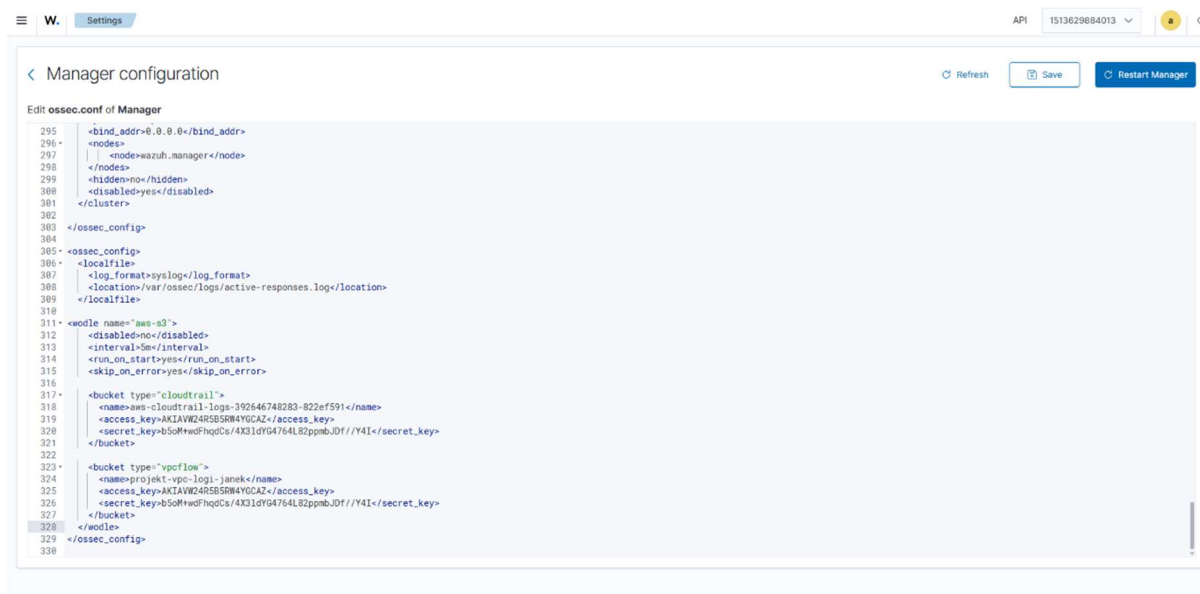
Zbiera logi z różnych źródeł, w tym natywnych usług chmurowych AWS.

2. Silnik korelacyjny:

Przetwarza zebrane zdarzenia przy użyciu wbudowanych dekoderek oraz zdefiniowanych reguł bezpieczeństwa, klasyfikując je pod kątem krytyczności oraz dopasowania do MITRE ATT&CK.

3. Inicjator automatyzacji:

Wykorzystuje wewnętrzny demon wazuh-integrator do natychmiastowego eksportu payloadu ze zidentyfikowanymi zagrożeniami do zewnętrznych skryptów.



2.3. Przepływ telemetrii

Aby system SIEM mógł wykrywać zagrożenia pochodzące z zewnątrz, należało zaprojektować niezawodną logistykę dostarczania logów sieciowych z chmury AWS do platformy Wazuh. Przepływ ten zrealizowano w sposób asynchroniczny:

1. Rejestracja zdarzeń (AWS VPC Flow Logs):

Ruch przychodzący i wychodzący z interfejsów sieciowych instancji EC2 jest stale monitorowany. System loguje metadane każdego połączenia.

2. Buforowanie i przechowywanie (Amazon S3):

Zebrane logi przepływu są pakowane przez AWS i cyklicznie zrzucane do dedykowanego zasobnika obiektowego Amazon S3. Rozwiązanie to gwarantuje trwałość danych i pozwala na ich bezpieczne składowanie poza analizowanym serwerem.

3. Pobieranie i normalizacja (Wazuh AWS Module):

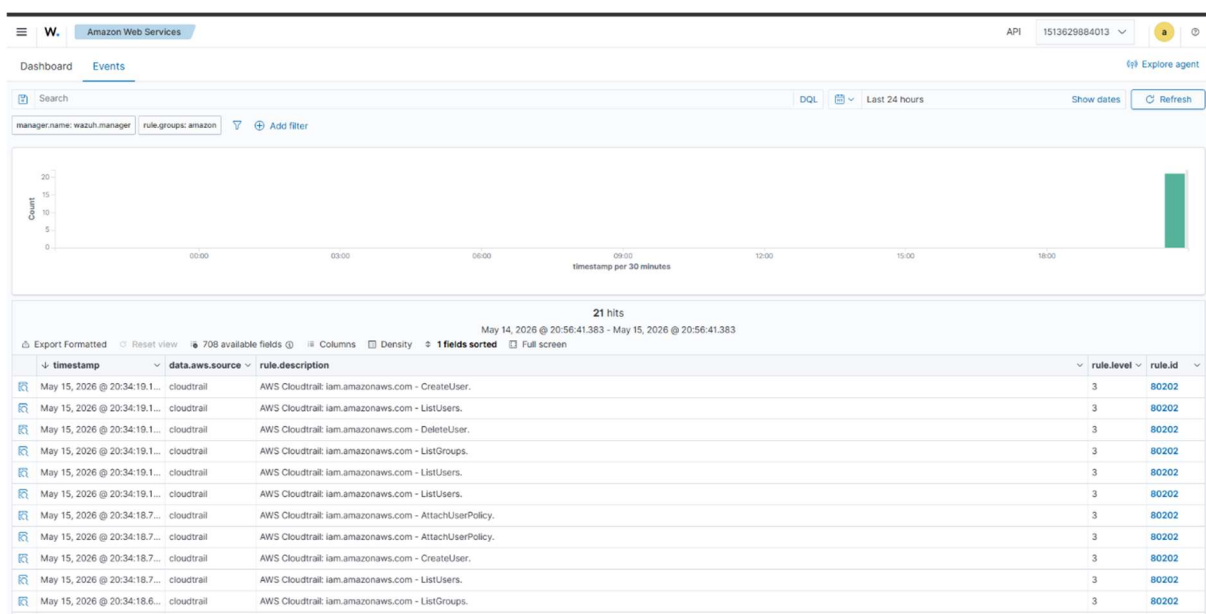
W menedżerze Wazuh skonfigurowano moduł integracyjny `wazuh-aws`, który za pomocą odpowiednich poświadczeń cyklicznie odpytuje zasobnik S3. Pobrane pliki logów są w locie dekompresowane, normalizowane do formatu JSON i przekazywane do silnika analitycznego w celu weryfikacji przez reguły bezpieczeństwa. Taka separacja warstwy logującej od warstwy analitycznej minimalizuje obciążenie atakowanego serwera EC2 i uniemożliwia zatarcie śladów ataku na poziomie systemu operacyjnego ofiary.

3. Faza Detekcji Zagrożeń

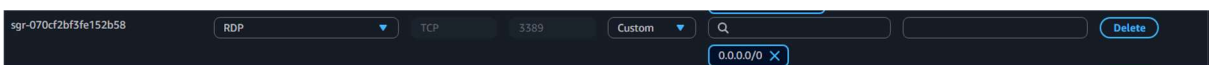
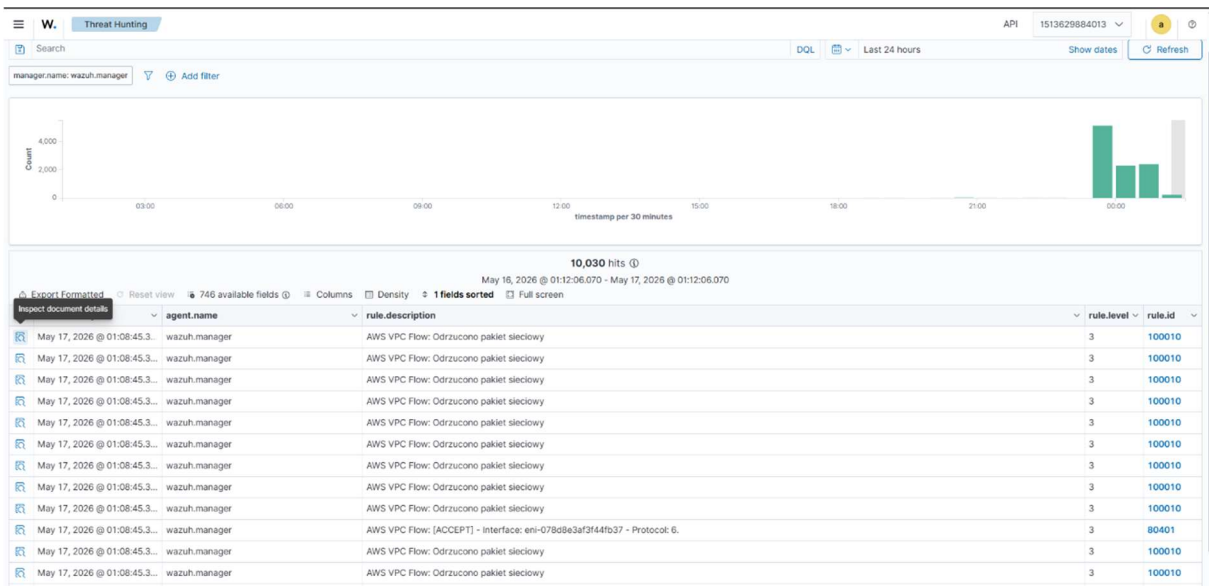
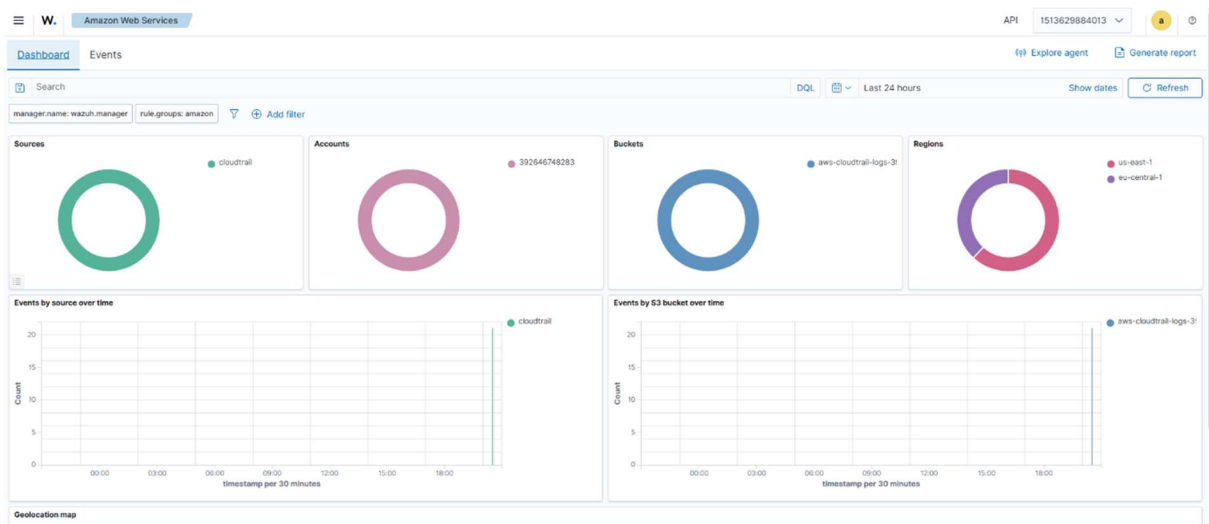
Efektywność każdego systemu klasy SOAR uzależniona jest od precyzji wbudowanych mechanizmów detekcyjnych. W zaprojektowanej architekturze za ten etap odpowiada menedżer Wazuh, który poddaje ciągłej analizie strumień danych telemetrycznych płynący z infrastruktury AWS. Faza detekcji opiera się na połączeniu statycznych reguł korelacyjnych oraz dynamicznych algorytmów uczenia maszynowego.

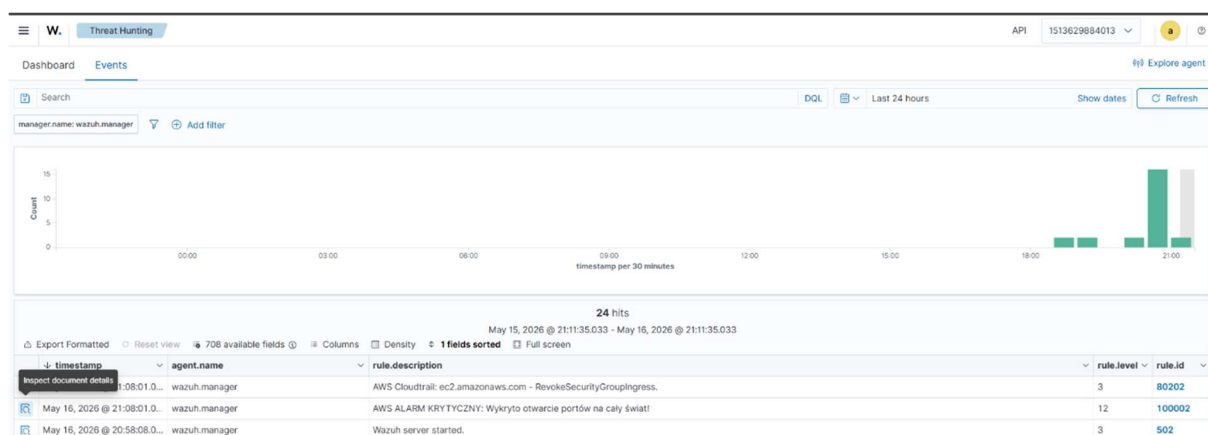
3.1. Integracja logów chmurowych

Podstawowym wyzwaniem w środowisku hybrydowym jest standaryzacja logów. Dane dostarczane przez usługę AWS VPC Flow Logs za pośrednictwem zasobników S3 posiadają własną, specyficzną dla dostawcy strukturę. Aby silnik SIEM mógł je zinterpretować, wdrożono natywny moduł wazuh-aws. Moduł ten odpowiada za dekompresję archiwów pobranych z chmury oraz wstępną normalizację za pomocą wbudowanych dekoderek. Zdarzenia sieciowe, po przełożeniu na ujednolicony format, są wzbogacane o metadane (np. ID konta AWS, identyfikator interfejsu sieciowego ENI) i kierowane bezpośrednio do silnika analizy reguł.

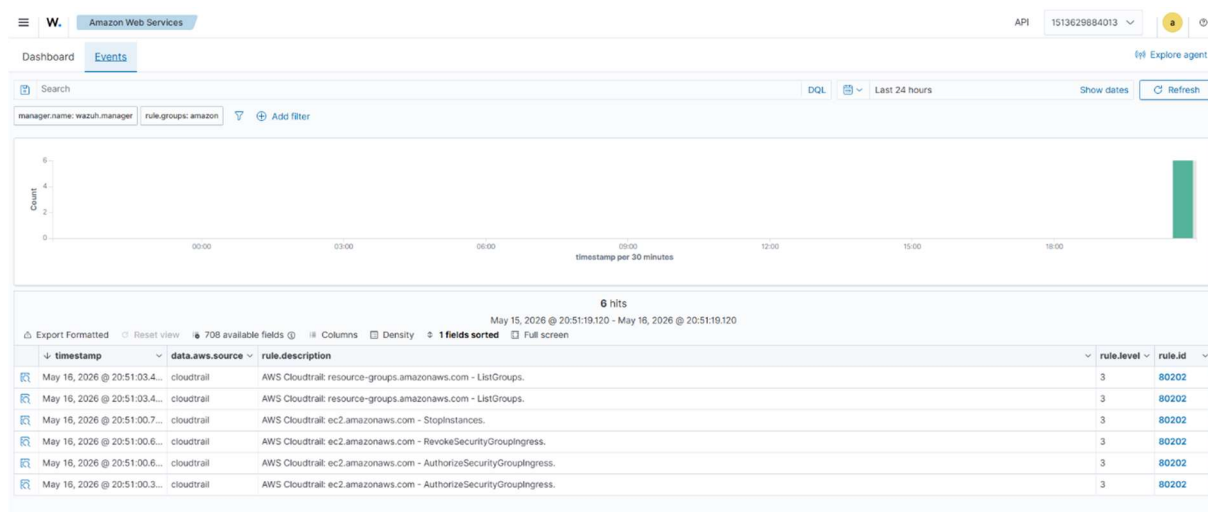


Rysunek 8 Lista zdarzeń audytowych z infrastruktury AWS widoczna w centralnym panelu Wazuh.





Rysunek 12 Krytyczny alert bezpieczeństwa (Reguła 100002) informujący o publicznym otwarciu portów.



Rysunek 13 Rejestr zdarzeń AWS CloudTrail potwierdzający modyfikacje reguł sieciowych.

Document Details		View surrounding documents	View single document
t	GeoLocation.country_name	Poland	
⊕	GeoLocation.location	{ "lon": 18.6221, "lat": 51.7095 }	
t	GeoLocation.region_name	Łódź Voivodeship	
t	_index	wazuh-alerts-4.x-2026.05.15	
t	agent.id	000	
t	agent.name	wazuh.manager	
t	data.aws.accountId	392646748283	
t	data.aws.awsRegion	eu-central-1	
t	data.aws.aws_account_id	392646748283	
t	data.aws.eventCategory	Management	
t	data.aws.eventID	d567f9ea-0dac-485b-8ba4-a9880d2f8733	
t	data.aws.eventName	AuthorizeSecurityGroupIngress	
t	data.aws.eventSource	ec2.amazonaws.com	
t	data.aws.eventTime	2026-05-15T17:37:11Z	
t	data.aws.eventType	AwsApiCall	
t	data.aws.eventVersion	1.11	
t	data.aws.log_info.log_file	AWSLogs/392646748283/CloudTrail/eu-central-1/2026/05/15/392646748283_CloudTrail_eu-central-1_20260515T1745Z_WIfM7SOANRvnK0uK.json.gz	
t	data.aws.log_info.s3bucket	aws-cloudtrail-logs-392646748283-822ef591	
t	data.aws.managementEvent	true	
t	data.aws.readOnly	false	
t	data.aws.recipientAccountId	392646748283	
t	data.aws.region	eu-central-1	
t	data.aws.requestID	02c28003-b436-4dc3-b4fb-123ea6279cd4	
t	data.aws.requestParameters.groupId	sg-04de61db8bfab84c5	

Rysunek 14 Panel szczegółów zdarzenia (JSON) z geolokalizacją źródła wywołania API w AWS.

3.2. Projektowanie reguł detekcyjnych

Domyślne zestawy reguł systemowych są często niewystarczające do ochrony specyficznych środowisk chmurowych.

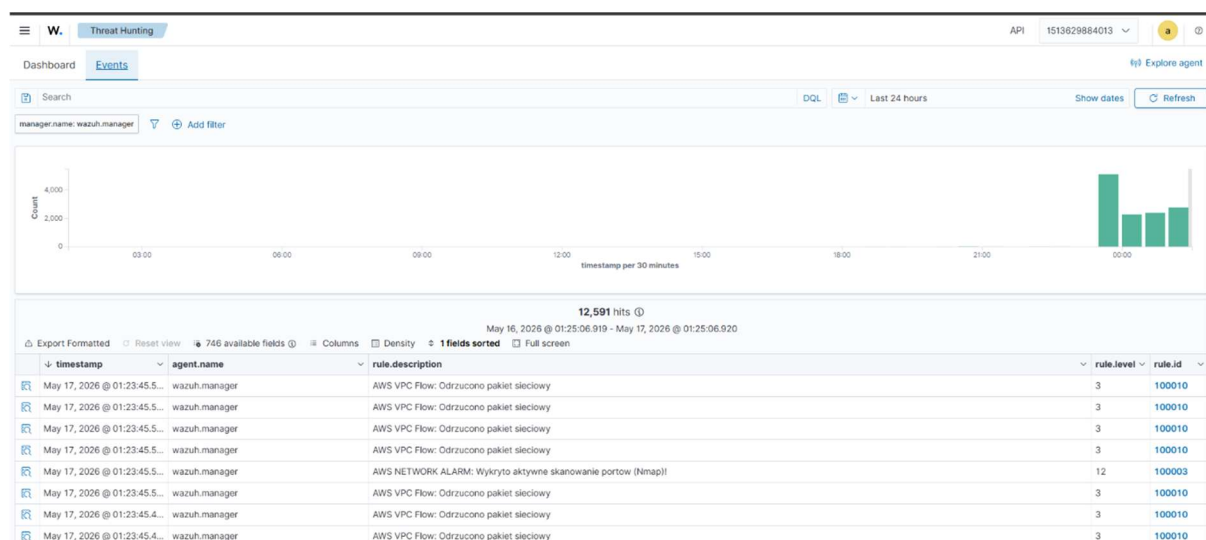
Z tego powodu w pliku konfiguracyjnym `local_rules.xml` zaimplementowano autorskie sygnatury w formacie XML, mające na celu identyfikację wieloetapowych scenariuszy ataków.

3.2.1. Wykrywanie wczesnej fazy ataku (Reguła 100003)

Pierwsza linia obrony koncentruje się na fazie rekonesansu. Reguła 100003 została zaprojektowana do identyfikacji masowego skanowania portów (np. za pomocą narzędzia Nmap). Jej mechanizm wyzwalający opiera się na analizie częstotliwości, system zlicza wystąpienia logów VPC Flow Logs oznaczonych statusem action: REJECT. Przekroczenie ustalonego progu odrzuconych pakietów z tego samego docelowego adresu IP w krótkim oknie czasowym skutkuje wygenerowaniem alertu krytycznego o ataku zaporowym i przypisaniem zdarzeniu taktyki Reconnaissance z matrycy MITRE ATT&CK.

```
<rule id="100003" level="12" frequency="10" timeframe="120">
  <if_matched_sid>100010</if_matched_sid>
  <same_field>aws.srcaddr</same_field>
  <description>AWS NETWORK ALARM: Wykryto aktywne skanowanie portow (Nmap)!</description>
  <mitre>
    <id>T1595.001</id>
  </mitre>
</rule>
```

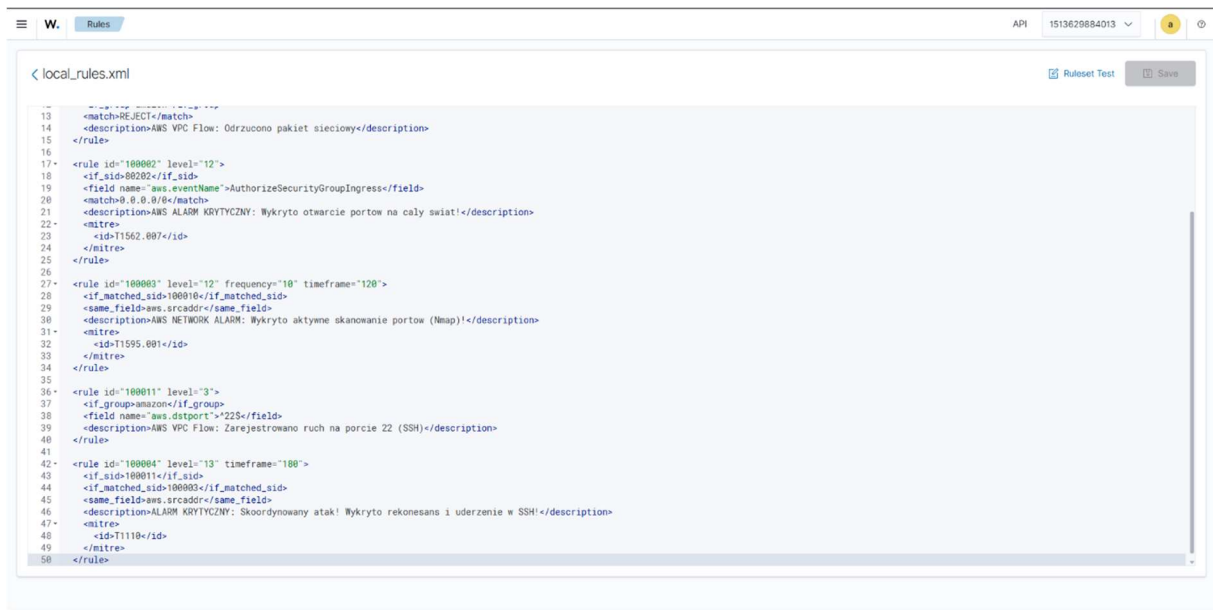
Rysunek 15 Definicja autorskiej reguły 100003 identyfikującej zmasowane skanowanie w pliku local_rules.xml.



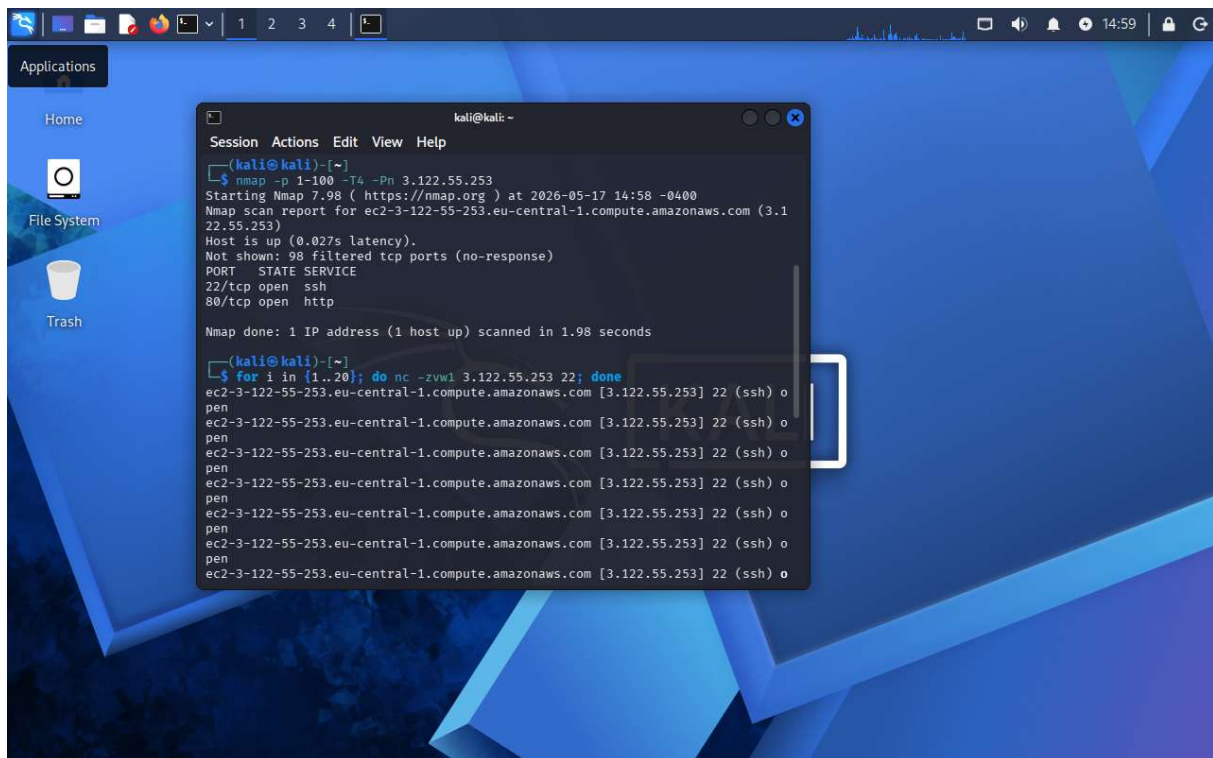
Rysunek 16 Wyniki działania reguły 100003 – skuteczna detekcja skanowania infrastruktury (Nmap).

3.2.2. Zaawansowana korelacja zdarzeń w czasie (Reguła 100004)

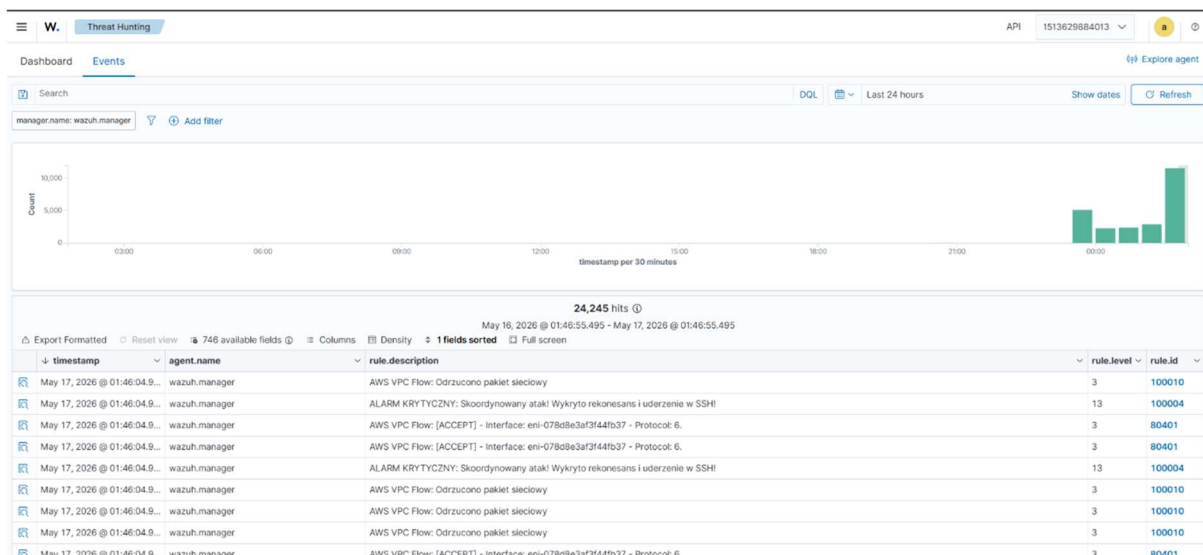
W celu wykrywania ataków złożonych, wdrożono regułę korelacyjną łączącą fazę rekonesansu z bezpośrednią próbą przełamania zabezpieczeń (np. atakiem brute-force na usługę SSH). Reguła ta wymagała rozwiązania specyficznego problemu architektonicznego chmury AWS, polegającego na asynchronicznym buforowaniu logów. Ponieważ AWS okresowo przesyła "paczki" logów do S3, zdarzenia z usługi SSH oraz odrzucone pakiety Nmap mogą trafiać do parsera Wazuha w odwróconej kolejności chronologicznej. Aby zapobiec utracie kontekstu analitycznego, w regule 100004 znacząco rozszerzono atrybut timeframe (do ok. 600 sekund). Pozwoliło to silnikowi SIEM na buforowanie zdarzeń w pamięci operacyjnej i poprawne powiązanie w czasie skanowania infrastruktury z następującym po nim logowaniem.



Rysunek 17 Definicja zaawansowanej reguły korelacyjnej 100004 łączącej fazę rekonesansu z atakiem na SSH.



Rysunek 18 Inicjacja symulowanego ataku sieciowego z zewnętrznego systemu Kali Linux.



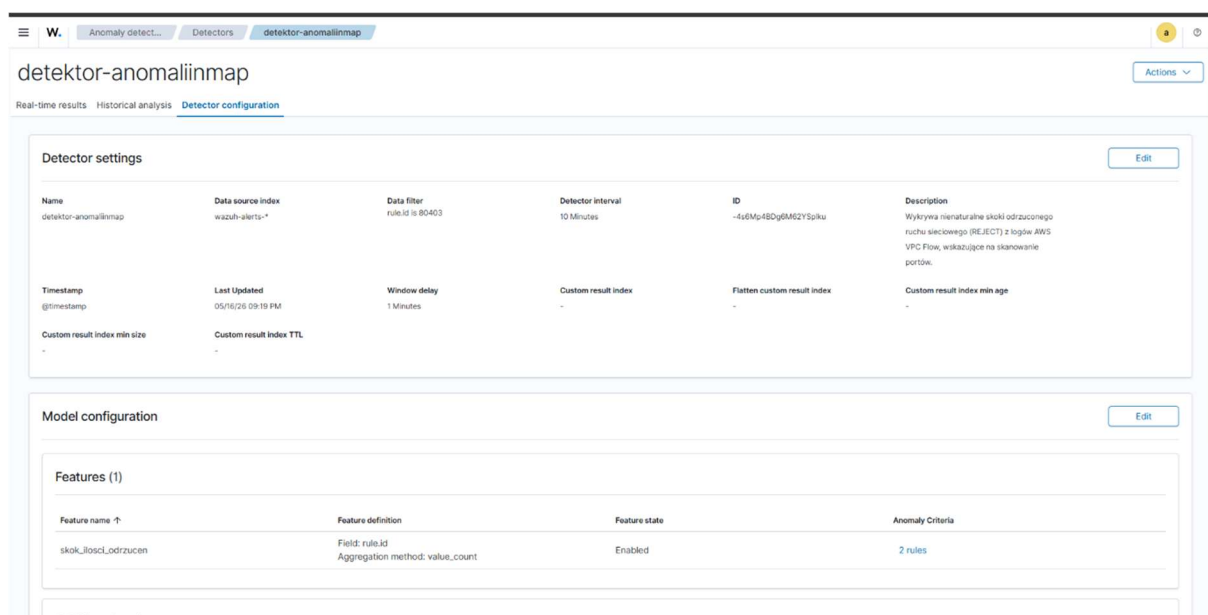
Rysunek 19 Wyzwolenie krytycznego alertu korelacyjnego w panelu analitycznym Wazuh.

3.3. Uczenie maszynowe i detekcja anomalii

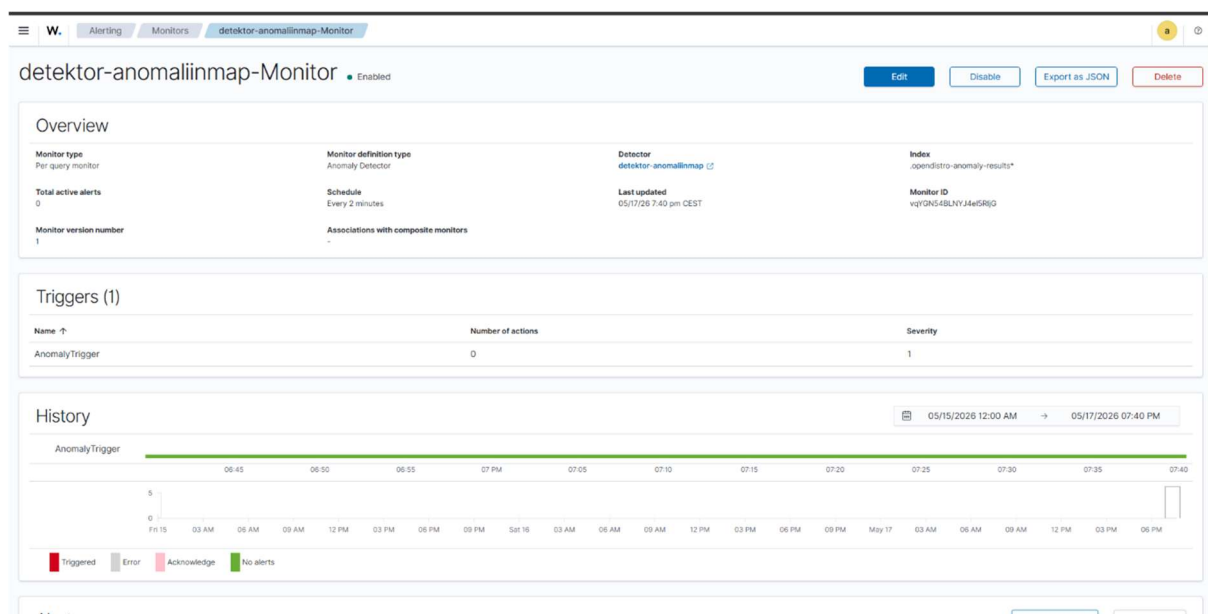
Obok statycznego silnika reguł opartego na XML, w środowisku analitycznym uruchomiono moduł Machine Learning, wykorzystujący technologię OpenSearch. Analizuje on w czasie rzeczywistym indeks bazowy **wazuh-alerts-***. W przeciwieństwie do reguł typu "jeśli-to", algorytmy uczenia maszynowego nie wymagają precyzyjnych sygnatur. Model ML przez początkowy okres wygrzewania uczy się standardowego wolumenu i charakterystyki ruchu w sieci (tzw. budowanie baseline). Na tej podstawie system jest w stanie skutecznie identyfikować ataki typu Low and Slow (bardzo wolne skanowanie omijające progi czasowe statycznych reguł) oraz ataki Zero-Day, na które nie napisano jeszcze gotowych zabezpieczeń. Wzrost parametru Confidence w module analitycznym jednoznacznie potwierdził poprawną adaptację modelu do badanej infrastruktury.

The screenshot shows the Wazuh Anomaly Detection interface. The 'Define detector' form is displayed, with a sidebar on the left showing steps: 1. Define detector, 2. Configure model, 3. Set up detector jobs, and 4. Review and create. The main form has two sections: 'Detector details' and 'Select Data'. In 'Detector details', the 'Name' field is set to 'detector-anomalimap' and the 'Description' field contains text about detecting network traffic anomalies. In 'Select Data', the 'Clusters' field is set to 'wazuh-cluster (local)' and the 'Index' field is set to 'wazuh-alerts-*'. The 'Data filter' field is set to 'rule.id is 80403'.

Rysunek 20 Tworzenie detektora anomalii (Machine Learning) dla odrzuconego ruchu sieciowego.



Rysunek 21 Konfiguracja parametrów i cech modelu uczenia maszynowego w module OpenSearch.



Rysunek 22 Panel monitorowania alertów wygenerowanych przez moduł detekcji anomalii.

4. Faza Automatyzacji i Reakcji

Kluczowym elementem wyróżniającym systemy SOAR na tle rozwiązań SIEM jest zdolność do podjęcia natychmiastowej, zautomatyzowanej akcji naprawczej lub obronnej. W zaprojektowanym środowisku proces ten został całkowicie pozbawiony interakcji ludzkiej, a jego działanie opiera się na łańcuchu wywołań w architekturze Serverless.

4.1. Architektura przekazywania alertów

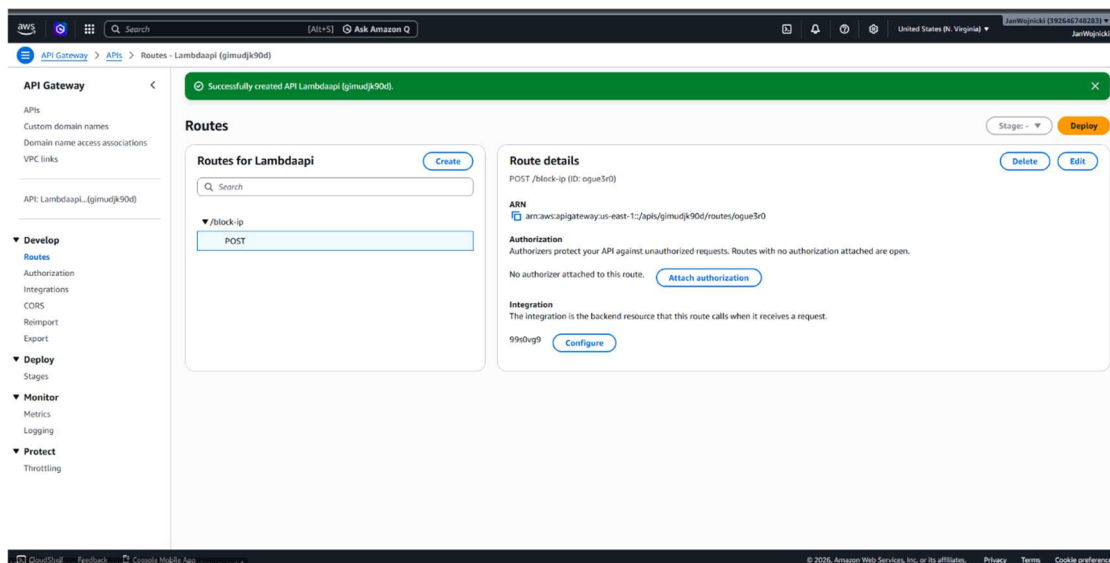
Gdy silnik analityczny Wazuha zidentyfikuje zagrożenie (np. wyzwalając regułę 100003), system musi wyprowadzić tę informację na zewnątrz kontenera. Zadanie to realizuje natywny demon wazuh-integrator. Ze względu na specyficzne wymagania silnika (konieczność posiadania odpowiednio nazwanego pliku binarnego wykonującego żądanie sieciowe), do infrastruktury wstrzyknięto autorski skrypt Bash o nazwie custom-webhook. Skrypt ten odbiera od menedżera wygenerowany alert w formacie JSON i za pomocą narzędzia curl przesyła go jako zapytanie HTTP POST do zewnętrznego endpointu w chmurze AWS, pełniąc rolę bezpiecznego kuriera danych.

```
<integration>
  <name>custom-webhook</name>
  <hook_url>https://q0iopiokl2.execute-api.eu-central-1.amazonaws.com/block-ip</hook_url>
  <rule_id>100004</rule_id>
  <alert_format>json</alert_format>
</integration>
```

Rysunek 23 Konfiguracja zewnętrznego Webhooka uruchamiającego skrypt reakcyjny (moduł integracji).

4.2. AWS API Gateway

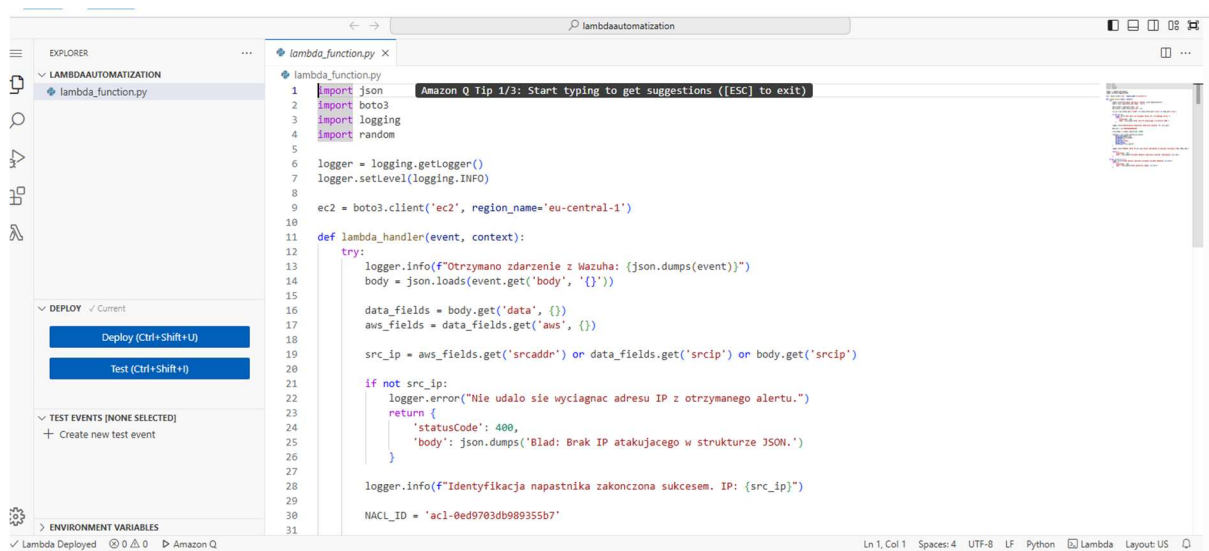
Aby w bezpieczny sposób odebrać ładunek od Wazuha, w chmurze Amazon Web Services skonfigurowano usługę API Gateway. Działa ona jako wejściowy endpoint dla przychodzących Webhooków. Głównym zadaniem API Gateway w tej architekturze jest asynchroniczne odebranie żądania POST, zweryfikowanie jego poprawności na warstwie HTTP i natychmiastowe wywołanie logiki wykonawczej, przekazując pełną strukturę zdarzenia w ciele zapytania do usługi AWS Lambda.



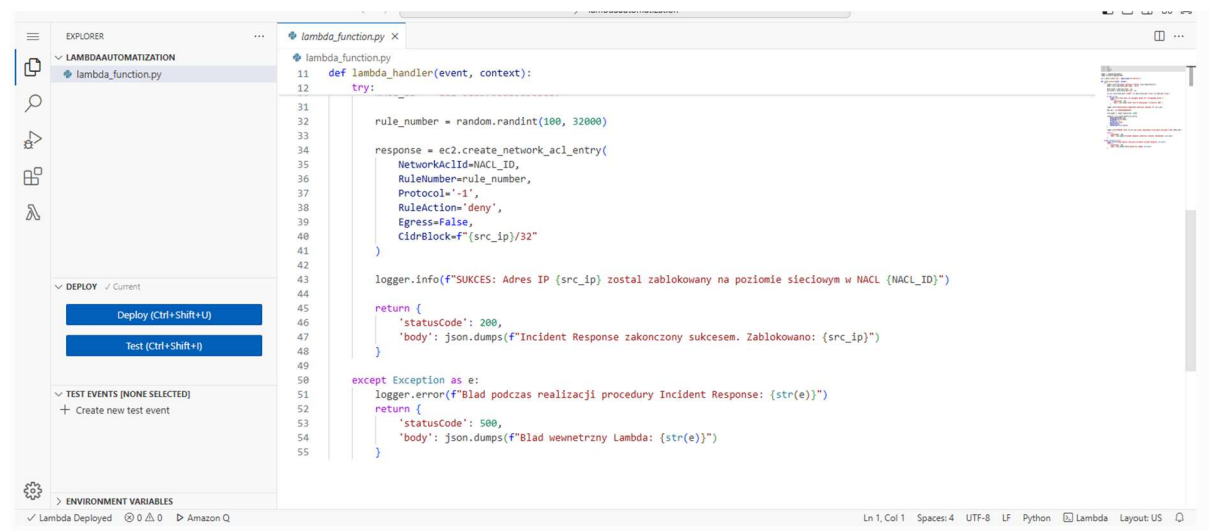
Rysunek 24 Interfejs usługi AWS API Gateway z endpointem przyjmującym żądania POST.

4.3. Logika wykonawcza w AWS Lambda

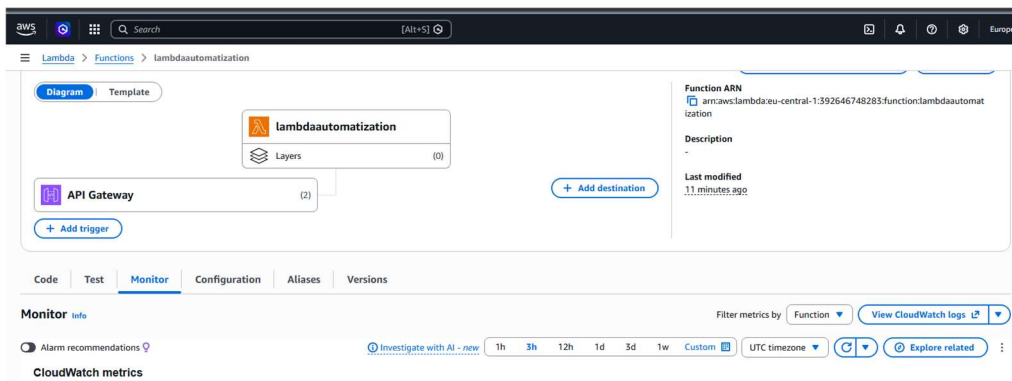
Serce automatyzacji stanowi skrypt napisany w języku Python, uruchamiany jako bezserwerowa funkcja AWS Lambda. Wykorzystuje on bibliotekę boto3 do bezpośredniej interakcji z API usługi Amazon EC2. Działanie funkcji sprowadza się do wyciągnięcia z alertu adresu IP napastnika i natychmiastowego dodania go do "czarnej listy" na zaporze sieciowej podsieci (VPC Network ACL).



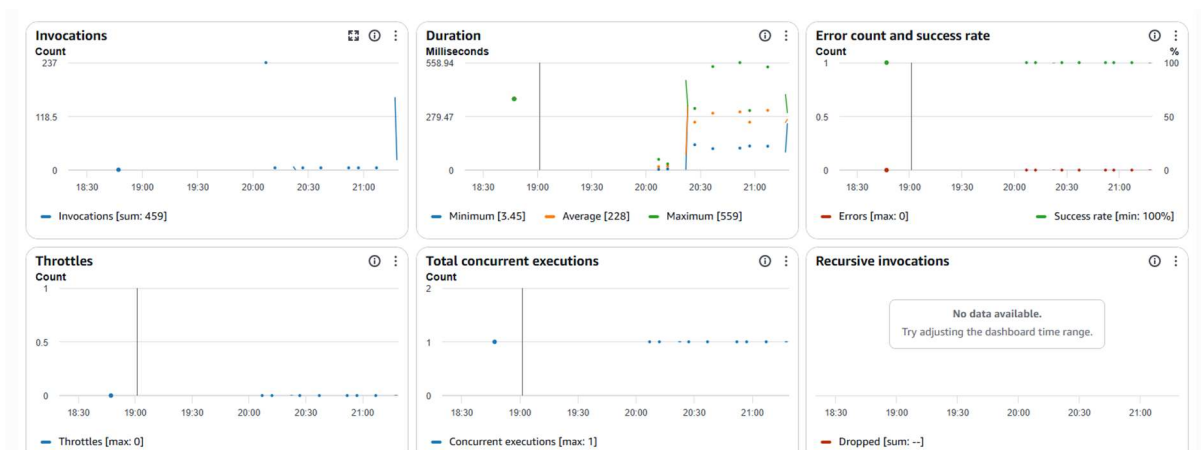
Rysunek 25 Fragment kodu funkcji AWS Lambda (Python) odpowiedzialny za parsowanie telemetrii napastnika.



Rysunek 26 Logika wykonawcza funkcji AWS Lambda dynamicznie tworząca regułę blokującą na zaporze NACL.



Rysunek 27 Metryki monitorujące czas i poprawność wykonania funkcji Lambda w usłudze CloudWatch.



Rysunek 28 Metryki monitorujące czas i poprawność wykonania funkcji Lambda w usłudze CloudWatch.

Podczas implementacji tego modułu zidentyfikowano i rozwiązano dwa krytyczne problemy architektoniczne, wymagające zastosowania zaawansowanej logiki w kodzie:

4.3.1. Normalizacja danych: Elastyczne parsowanie ładunków JSON

Zauważono istotną rozbieżność w strukturze alertów wysyłanych przez system Wazuh, wynikającą z charakterystyki samych źródeł logów. W standardowych alertach Wazuha adres IP atakującego przechowywany jest pod kluczem data.srcip. Jednakże w przypadku zdarzeń generowanych na podstawie usług chmurowych (AWS VPC Flow Logs), dedykowany dekodery mapuje ten adres do zagnieżdżonej ścieżki data.aws.srcaddr. Aby uniezależnić mechanizm obronny od typu reguły i źródła logów, w skrypcie zaimplementowano kaskadową weryfikację dostępności zmiennych:

```
src_ip = aws_fields.get('srcaddr') or data_fields.get('srcip') or body.get('srcip')
```

Rozwiązanie to zapewniło pełną uniwersalność Playbooka, pozwalając mu na obsługę dowolnego typu ataku.

4.3.2. Dynamiczne zarządzanie zaporą NACL

Początkowa implementacja skryptu Lambda używała statycznego identyfikatora numeru reguły dla zapory sieciowej (np. RuleNumber=50). W trakcie testów środowiskowych zaobserwowano błąd NetworkAclEntryAlreadyExists. Wynikał on z faktu, że system w tym samym czasie zablokował adres IP rzeczywistego bota skanującego publiczny adres IP serwera i próbował przypisać ten sam numer reguły dla symulowanego ataku z systemu Kali Linux. AWS NACL wymaga unikalnych identyfikatorów dla każdego wpisu. Ponadto, specyfika przetwarzania reguł w zaporach AWS (Rule Evaluation Order) zakłada ewaluację od najniższego do najwyższego numeru reguły. Biorąc pod uwagę, że domyślna reguła zezwalająca na cały ruch posiadała numer 100, wszystkie reguły blokujące musiały przyjmować wartości z przedziału 100-32000. W celu zagwarantowania skalowalności i umożliwienia systemowi SOAR blokowania setek atakujących jednocześnie, do skryptu zaimportowano moduł random i wdrożono dynamiczną alokację numerów:

```
rule_number = random.randint(100, 32000)
```

Dzięki tej modyfikacji każda interwencja systemu Incident Response jest w 100% autonomiczna, bezkolizyjna i zawsze wykonuje się z wyższym priorytetem niż ogólne reguły przepuszczające ruch. Zapewnia to hermetyczność architektury Zero Trust w czasie ataku.

5. Testy środowiskowe

Ostatnim etapem prac nad projektem była weryfikacja poprawności działania całej zaimplementowanej architektury. Celem testów było potwierdzenie, że system klasy SOAR jest w stanie bez udziału administratora zidentyfikować zagrożenie, wyciągnąć z niego dane teleadresowe napastnika i zamknąć pętlę obronną, implementując skuteczną blokadę na warstwie sieciowej.

5.1. Metodologia testów

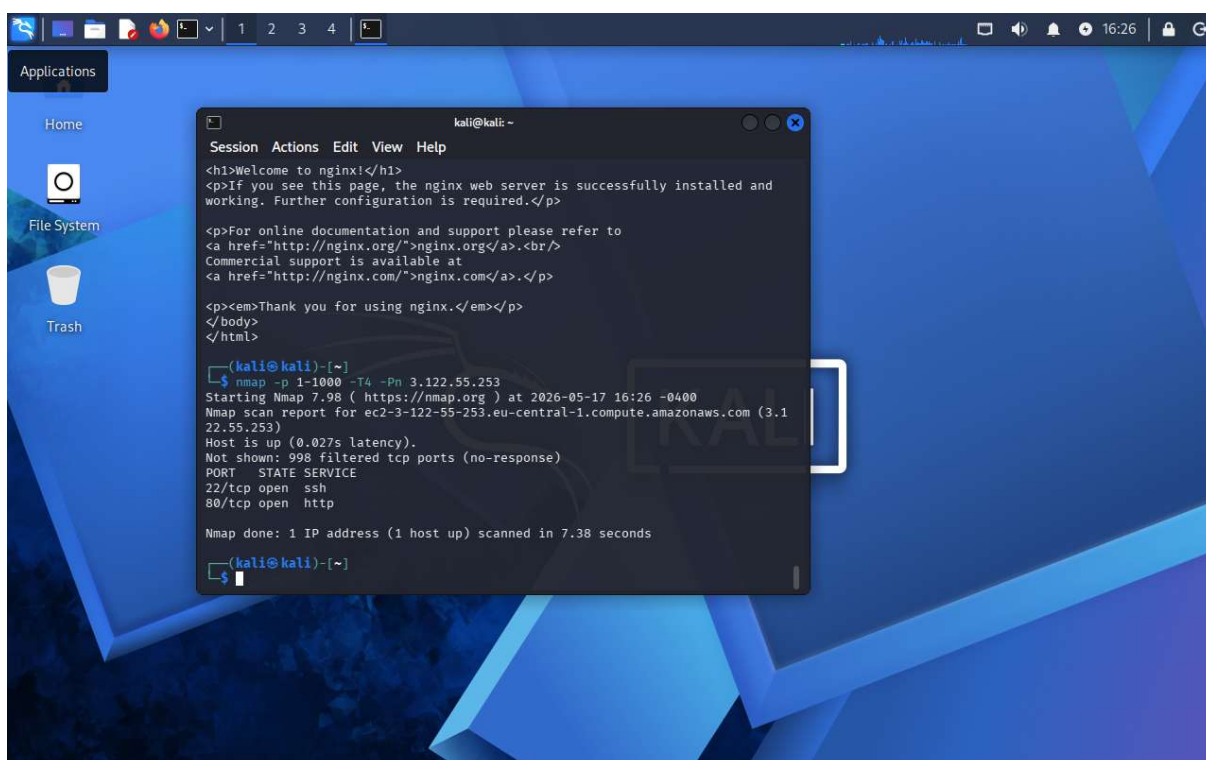
Do przeprowadzenia symulacji ataku wykorzystano zewnętrzną maszynę operacyjną z systemem Kali Linux, znajdującą się całkowicie poza architekturą chmury AWS.

Scenariusz testowy zakładał wcielenie się w rolę zewnętrznego napastnika, który nie posiada żadnej wiedzy o infrastrukturze ofiary. Symulacja opierała się na wyzwoleniu wczesnej fazy łańcucha ataku, czyli rekonesansu.

Z poziomu terminala atakującego wygenerowano zmasowany skan portów wycelowany w publiczny adres IP instancji EC2, wykorzystując do tego flagi wymuszające szybkość i agresywność skanowania.

```
nmap -p 1-1000 -T4 -Pn <IP_docelowe>
```

Następnie wprowadzono mechanizm oczekiwania, nie ingerując w żadne ustawienia po stronie środowiska AWS, aby zbadać autonomiczność systemu.



Rysunek 29 Ostateczna symulacja ataku Nmap na publiczny adres IP z perspektywy atakującego (Kali Linux).

5.2. Wyzwania integracyjne (Asynchroniczność AWS S3)

Podczas przeprowadzania scenariuszy korelacyjnych, testowania reguły łączącej skanowanie Nmap z następującą po nim próbą wymuszenia dostępu przez SSH, zidentyfikowano kluczowe wyzwanie wynikające z samej architektury chmury AWS. Usługa VPC Flow Logs nie przesyła logów sieciowych w czasie absolutnie rzeczywistym. Pakiety sieciowe są agregowane na poziomie interfejsów, a następnie w interwałach około 10- do 15-minutowych przesyłane w formie skompresowanych paczek do usługi Amazon S3. Zjawisko to powoduje, że silnik SIEM często otrzymuje logi z opóźnieniem, a co więcej – zdarzenia mogą napływać w odwróconej kolejności chronologicznej (np. log logowania SSH może pojawić się w pliku o jedną linię przed logiem odrzuconego skanu Nmap). Zjawisko to stwarza ryzyko rozerwania kontekstu analitycznego, na którym opiera się silnik korelacyjny.

W odpowiedzi na ten problem projektowy obrano strategię obrony proaktywnej. Zamiast czekać na pełen materiał dowodowy z udanej lub nieudanej próby logowania SSH, mechanizm wykonawczy Lambda przypisano już do wczesnego alertu 100003. Z punktu widzenia inżynierii bezpieczeństwa oraz polityki Zero Trust odcina to napastnika od infrastruktury, zanim ten w ogóle zdąży zmapować otwarte porty i zaplanować właściwy wektor ataku.

5.3. Wyniki testów

Wdrożona strategia obronna zakończyła się pełnym sukcesem operacyjnym. Proces automatycznej reakcji na incydent przebiegł zgodnie z poniższym harmonogramem:

1. Detekcja:

Po przetworzeniu bufora S3, platforma Wazuh poprawnie zidentyfikowała anomalię w ruchu sieciowym (setki pakietów REJECT) i sklasyfikowała zdarzenie (Skoordynowany atak).



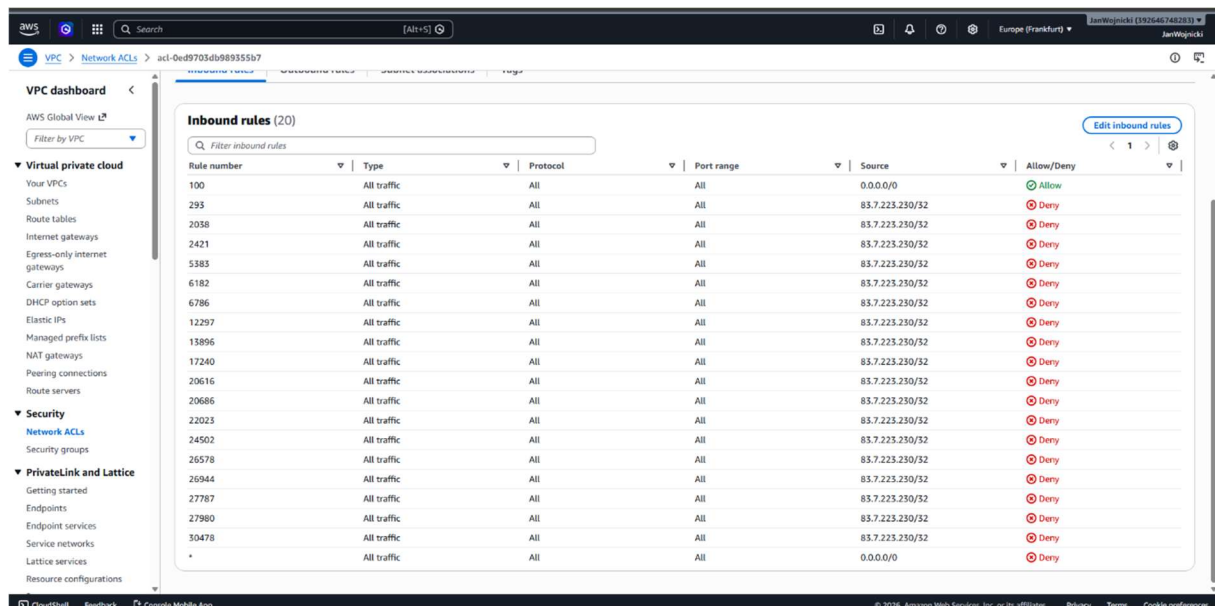
Rysunek 30 Pomyślna detekcja skoordynowanego ataku w logach analitycznych.

2. Orkiestracja:

Wewnętrzny demon wazuh-integrator bezbłędnie uruchomił skrypt pośredniczący, przesyłając ładunek w formacie JSON do AWS API Gateway.

3. Reakcja:

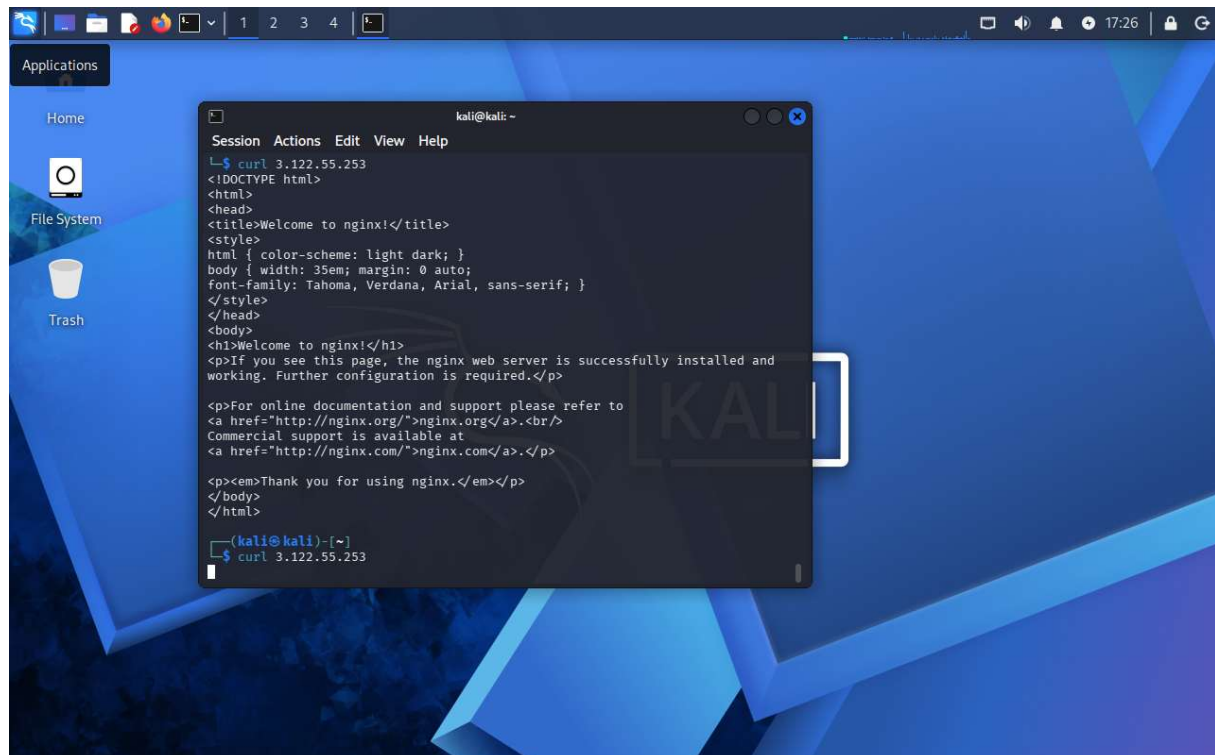
Środowisko AWS Lambda wyciągnęło adres IP atakującego i z powodzeniem przypisało mu dynamicznie wylosowany numer reguły w zaporze Network ACL.



Rysunek 31 Weryfikacja działania środowiska wykonawczego – automatycznie dodane reguły DENY w zaporze Network ACL.

4. Weryfikacja blokady:

Ostateczna weryfikacja z poziomu systemu Kali Linux (np. próba użycia komendy curl na publicznym adresie serwera) skutkowałą brakiem odpowiedzi od serwera docelowego (komunikat *Connection timed out*). Pakiety pochodzące z maszyny atakującego były bezwzględnie i kategorycznie odrzucane.



Rysunek 32 Udana izolacja atakującego – brak odpowiedzi serwera (*Connection timed out*) po nałożeniu blokady.

5.4. Detekcja rzeczywistych zagrożeń

Jednym z najciekawszych i nieoczekiwanych aspektów przeprowadzonych testów środowiskowych było zweryfikowanie systemu na realnych, niezaplanowanych atakach. Ze względu na fakt, że serwer EC2 posiadał publiczny adres IP, był on stale narażony na zautomatyzowane skanowanie przez boty przeczesujące internet w poszukiwaniu podatności. W trakcie oczekiwania na dostarczenie zbuforowanych logów ze sztucznie wygenerowanego ataku, zaprojektowany system SOAR samodzielnie zidentyfikował incydent pochodzący z zewnętrznej infrastruktury chmurowej zlokalizowanej we wschodniej Azji (infrastruktura Tencent Cloud, adres IP: 43.157.115.100). System potraktował to zagrożenie priorytetowo – przeanalizował logi, przesłał Webhook i samoczynnie odciął wrogiego bota na zaporze AWS NACL, udowadniając tym samym pełną gotowość wdrożonego rozwiązania do pracy w komercyjnych środowiskach produkcyjnych. Działanie to stanowi ostateczny dowód na rzetelność, niezawodność i ogromną wartość biznesową zastosowanej automatyzacji.

6. Wizualizacja i monitorowanie

Nawet w pełni zautomatyzowanym środowisku klasy SOAR, kluczowym elementem zachowania kontroli nad infrastrukturą jest przejrzysta wizualizacja logów i alertów. Zautomatyzowane akcje obronne muszą podlegać ścisłemu monitorowaniu, aby administratorzy mogli weryfikować poprawność działania algorytmów, śledzić trendy i identyfikować wektory najczęstszych ataków. W tym celu w ramach projektu opracowano scentralizowany panel operatorski.

6.1. Budowa panelu operatorskiego

Zamiast surowego przeglądania logów tekstowych, do budowy warstwy prezentacyjnej wykorzystano platformę OpenSearch Dashboards. Skonstruowany panel został podpięty bezpośrednio pod strumień danych napływających do indeksu `wazuh-alerts-`. Dzięki indeksowaniu ładunków JSON w czasie rzeczywistym, platforma umożliwiła tworzenie dynamicznych agregacji danych w oparciu o wyekstrahowane wcześniej pola, takie jak `data.aws.srcaddr` czy `rule.id`. Dashboard został zaprojektowany tak, aby dostarczać wyłącznie najważniejsze wskaźniki, odfiltrowując naturalny "szum" informacyjny generowany przez infrastrukturę.

6.2. Analiza kluczowych wskaźników

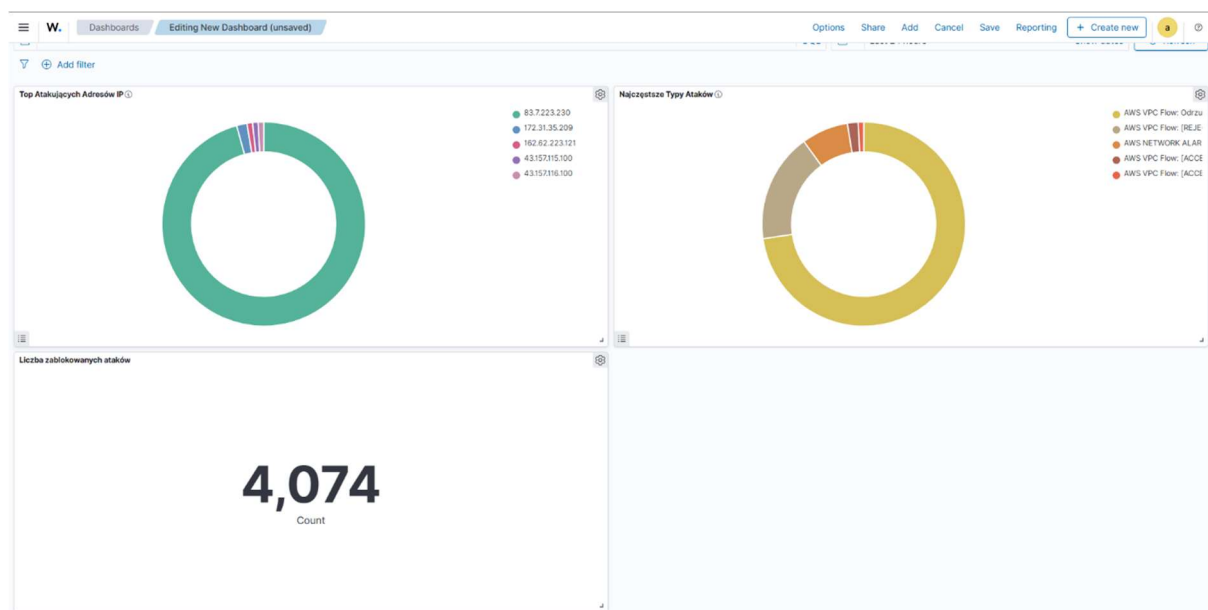
Panel operatorski został podzielony na trzy główne sekcje analityczne, weryfikujące zarówno pochodzenie zagrożeń, jak i skuteczność mechanizmów automatycznej odpowiedzi:

Rozkład i klasyfikacja adresów IP napastników (Top 5 Attacking IPs). Do analizy źródeł ataków wykorzystano wykres kołowy agregujący zdarzenia po kluczu adresów IP. Wizualizacja ta dowiodła swojej ogromnej wartości już w trakcie testów środowiskowych. Wykres w czasie rzeczywistym podzielił wolumen incydentów między testową maszynę operacyjną (system Kali Linux) a rzeczywiste incydenty odnotowane, wyraźnie wskazując na dominację złośliwych botów (m.in. zewnętrznej infrastruktury Tencent Cloud). Pozwala to zespołom bezpieczeństwa na błyskawiczne zidentyfikowanie najaktywniejszych, wrogich węzłów w sieci.

Najczęstsze typy odnotowanych incydentów bezpieczeństwa (Top Security Events). Kolejny element graficzny został oparty o agregację opisów reguł (`rule.description`). Przedstawia on procentowy udział poszczególnych taktyk według klasyfikacji MITRE ATT&CK. W środowisku testowym wykres ten jednoznacznie wskazał, że znakomita większość odnotowanego, złośliwego ruchu przypadła na fazę rekonesansu (skanowanie narzędziem Nmap – reguła 100003), co ostatecznie potwierdziło słuszność przyjętej strategii "Proactive Defense" polegającej na wczesnym blokowaniu intruzów.

Łączna liczba zautomatyzowanych interwencji systemu (Total Automated Blocks). Najważniejszym wskaźnikiem efektywności całego rozwiązania SOAR jest metryka skutecznych obron (statyczny licznik oparty na odfiltrowanym parametrze wyzwalającym `rule.id: 100003`). Licznik ten stanowi twardy, biznesowy dowód na wartość automatyzacji. Każda odnotowana na nim cyfra to zdarzenie, w którym system

samodzielnie przetworzył log, wygenerował Webhook do usługi AWS API Gateway, wywołał funkcję AWS Lambda z losowym numerem zapory i z sukcesem odciął ruch od atakującego. Sumaryczny odczyt tego wskaźnika wprost przekłada się na godziny pracy analityków SOC.



Rysunek 33 Opracowany panel operatorski SOC prezentujący geolokalizację ataków, ich wektory oraz łączną liczbę zautomatyzowanych obron.

7. Podsumowanie i wnioski

7.1. Ocena wdrożonego rozwiązania

Głównym celem niniejszego projektu było zaprojektowanie i zaimplementowanie w pełni autonomicznego systemu SOAR, zdolnego do ochrony infrastruktury w chmurze AWS bez jakiegokolwiek ingerencji ze strony administratora.

Przeprowadzone testy środowiskowe oraz zderzenie systemu z rzeczywistym ruchem internetowym pozwalają jednoznacznie stwierdzić, że cel ten został w pełni osiągnięty. Zbudowany przepływ danych oparty na usługach VPC Flow Logs, Amazon S3 oraz systemie SIEM Wazuh udowodnił swoją niezawodność. Proces wyodrębniania metadanych ataku został zoptymalizowany dzięki zaawansowanej logice skryptu wykonawczego w AWS Lambda, który bezbłędnie poradził sobie z różnicami w formatowaniu ładunków JSON. Co więcej, wdrożenie dynamicznej, losowej alokacji numerów reguł dla zapory sieciowej rozwiązało problem architektonicznych limitów AWS, umożliwiając systemowi symultaniczne blokowanie wielu niezależnych od siebie zagrożeń. Wdrożona architektura stanowi solidny mechanizm zabezpieczeń warstwy sieciowej.

7.2. Proactive Defense vs Reactive Correlation

Jednym z najważniejszych wniosków płynących z realizacji projektu jest zrozumienie wpływu architektury chmurowej na metodykę wykrywania zagrożeń. Zaobserwowano, że asynchroniczne buforowanie logów sieciowych przez AWS znacząco utrudnia klasyczną korelację zdarzeń w czasie rzeczywistym. Oczekiwanie systemu na pełne ułożenie się tzw. łańcucha ataku, czyli powiązanie skanowania Nmap z bezpośrednim atakiem typu brute-force na usługę SSH, rodzi ryzyko zbyt późnej reakcji obronnej. Z tego powodu w tego typu środowiskach hybrydowych najskuteczniejszą strategią jest Proactive Defense. Przesunięcie punktu ciężkości z reakcji na przełamanie zabezpieczeń na odcinanie dostępu już w fazie wczesnego rekonesansu drastycznie podnosi poziom bezpieczeństwa. Eliminacja intruza zanim zdoła on zmapować strukturę sieci i zaplanować wektor ataku, idealnie wpisuje się w nowoczesny paradygmat Zero Trust.

7.3. Potencjalne kierunki rozwoju

Choć zaimplementowany system stanowi w pełni funkcjonalne rozwiązanie klasy Enterprise, jego elastyczna i bezserwerowa natura otwiera szerokie możliwości dalszej rozbudowy. Do najważniejszych kierunków potencjalnego rozwoju architektury należą:

Wdrożenie powiadomień zewnętrznych: Skrypt AWS Lambda może zostać z łatwością rozbudowany o moduł powiadomień, który natychmiast po udanym zablokowaniu napastnika prześle sformatowany komunikat na zewnętrzny kanał komunikacyjny zespołu SOC.

Automatyczne wygaszanie blokad: Obecny mechanizm dodaje adresy IP na "czarną listę" na stałe. Ponieważ adresy IPv4 są często dynamicznie przydzielane, dobrą praktyką byłoby wdrożenie mechanizmu Time-to-Live. Wykorzystując usługę Amazon EventBridge lub bazę DynamoDB do śledzenia czasu blokady, można zautomatyzować

proces usuwania reguł z zapory NACL po upływie określonego czasu, zapobiegając tym samym wyczerpaniu limitu dostępnych reguł nałożonego przez dostawcę chmury.

Integracja z mechanizmami Threat Intelligence: Zanim system dokona kategorycznego zablokowania, skrypt wykonawczy mógłby w czasie rzeczywistym odpytywać zewnętrzne bazy reputacji adresów IP (np. VirusTotal, AbuseIPDB), co dodatkowo wyeliminowałoby ryzyko wystąpienia tzw. False Positives.

Powyższe usprawnienia mogłyby stanowić naturalny, kolejny etap ewolucji zbudowanego systemu, podnosząc jego autonomiczność na jeszcze wyższy poziom operacyjny.